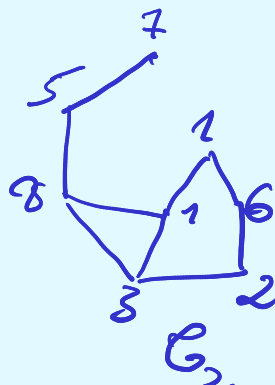
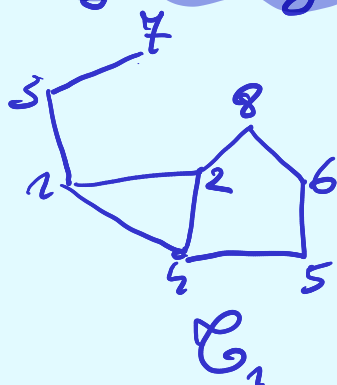


AiSD

WYKŁAD XI

SORTOWANIE-ZASTOSOWANIA

izomorfizm grafów



Grafy G_1, G_2 są izomorficzne, gdy (nieformalnie), gdy da się tak przekształcić wierzchołki G_2 , żeby dostać G_1 , formalnie

DEF $G_1 = (V_1, E_1)$ izomorficzny z $G_2 = (V_2, E_2)$

$\Leftrightarrow$

Istnieje permutacja $\pi: V_2 \rightarrow V_1$, że $V_1 = \{\pi[v]: v \in V_2\}$
 $E_1 = \{\{\pi[u], \pi[v]\}: \{u, v\} \in E_2\}$

Dla powyższego przykładu mamy $\pi[8] = 1, \pi[1] = 2$,
 $\pi[i] = i$ dla $i \neq 8, i \neq 1$.

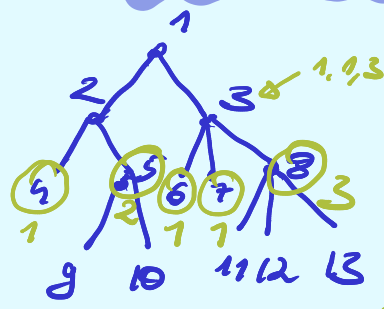
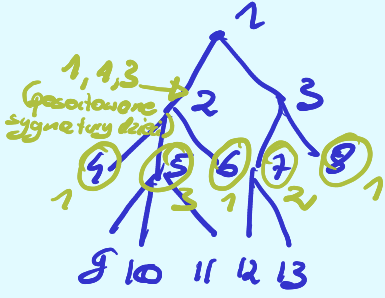
~~Problem jest NP-trudny.~~ Trudny, ale nie NP-Trudny (chyba).

izomorfizm drzew

Wejście: T_1, T_2 - dwa drzewa ekorenowane

Wyjście: TAK/NIE zależnie od tego, czy drzewa są izomorficzne

W definicji izomorfizmu dla drzew ukoszonych,
dodajemy dodatkowo, aby $\pi(\text{root}(T_1)) = \text{root}(T_2)$



Na gistoju
- sygnatury
(sygnatury 2
wiechołków
z równe $\Leftrightarrow$
moje tyle samo
dzieci)

Pomysł - $\forall$ v policzyć głębokość

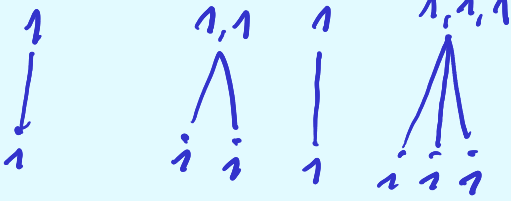
W listach przechowujemy wierzchołki tak, że
w k -tej liście mamy wierzchołki z k -tego
poziomu.

Signature:

Die listi:

$i \quad i \ i \ i \quad i \ i \ i$

Dolej



Sortujemy cieżgi sygnatur
dzieci. Jeżeli otrzymane ciężki
są równe (dla 2 wierszółków),
to przydzielamy tę samą
sygnaturę, lub inną z podobnych
pozytywkach.

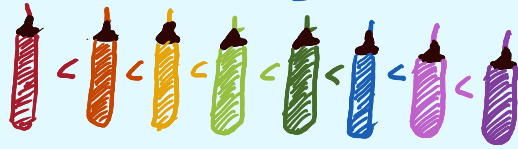


1234 $\rightarrow$ 10
 135 $\rightarrow$ 11
 135 $\rightarrow$ 11
 1910 $\rightarrow$ 12
 489 $\rightarrow$ 13
 7 $\rightarrow$ 14

Quick sort

mała ilustracja:-) Wiele razy było to tłumaczone na liściach, to teraz zrobimy na mazurkach.

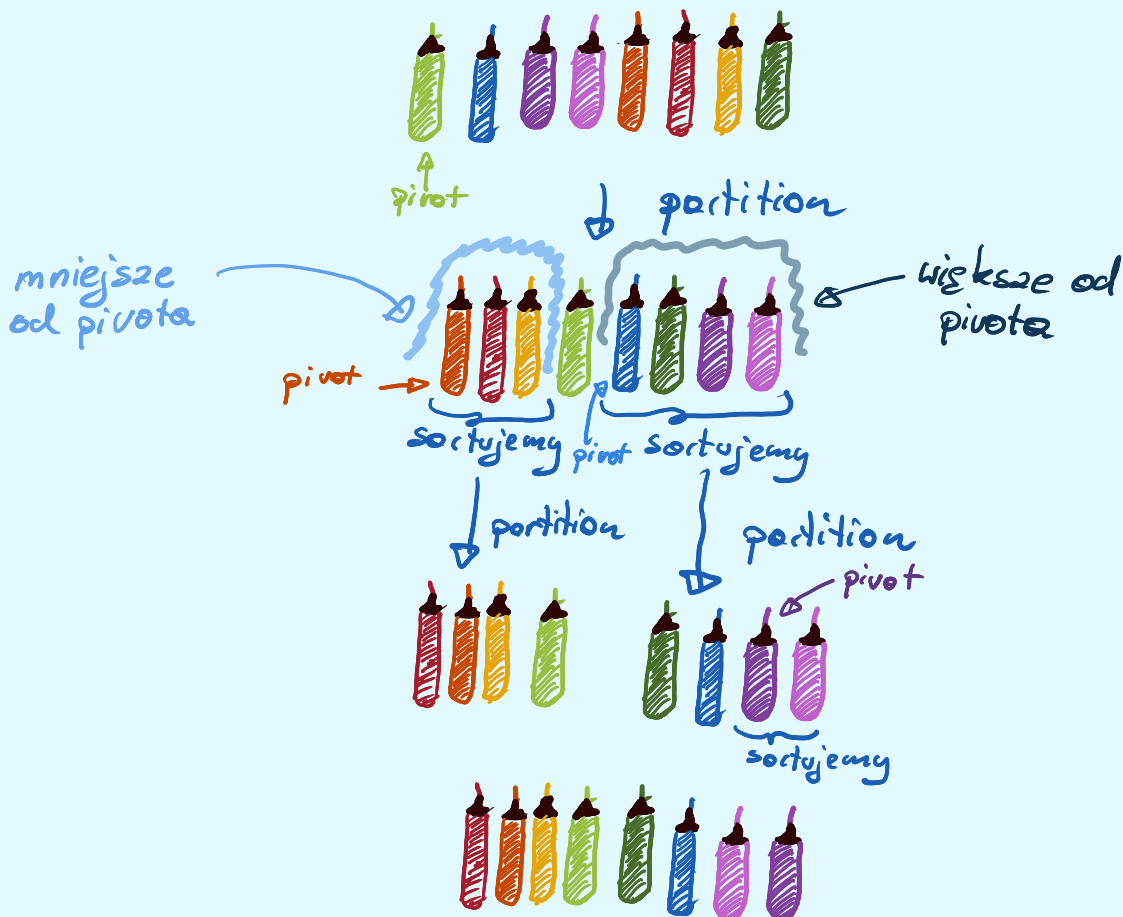
Porządek mazurków



W każdym wywołaniu wybieramy pivot i umieszczamy elementy mniejsze od niego na indeksach mniejszych, a większe na indeksach większych (partition).

Następnie sortujemy elementy na lewo i na prawo od pivotu.

TC - zależy od wyboru pivotu: Na razie wybieramy go jako pierwszy z brzoju.



Pseudokod

rekursja

QuickSort($A[b \dots e]$):

- if ($b < e$):
- $q \leftarrow \text{Partition}(A[b, \dots, e])$
- QuickSort($A[b, \dots, q-1]$)
- QuickSort($A[q+1, e]$)

Diagram annotations:

- Arrow from q to "początek i koniec sortowanego fragmentu"
- Arrow from q to "podziel elementy względem wybranego pivotu i zwróć jego indeks"

Partition można wykonać w miejscu. Szukamy kolejnych najmniejszych $i \geq b$ i kolejnych największych $j \leq e$, t. że $A[i] \geq p$ oraz $A[j] < p$ i zamieniamy miejscami

Partition ($A[b, \dots, e]$):
 x ← dowolny element z A (pivot)
 $i \leftarrow b$
 $j \leftarrow e$
 while ($j > i$):
 while $A[i] < x$ $i++$
 while $A[j] > x$ $j--$
 if $j > i$:
 $A[i] \leftrightarrow A[j]$
 $j--$
 $i++$
 return j

Poprawność pozostawiam
 jako ćwiczenie

UWAGA! Zgłosem uwagi z WDL

TC:

Pivota wybieramy losowo z $A[b \dots e]$ z rozkładem jednostajnym. Zakładamy, że elementy A są różne.

Niech $a_1, \dots, a_n$ oznacza wartości z A posortowane rosnąco.

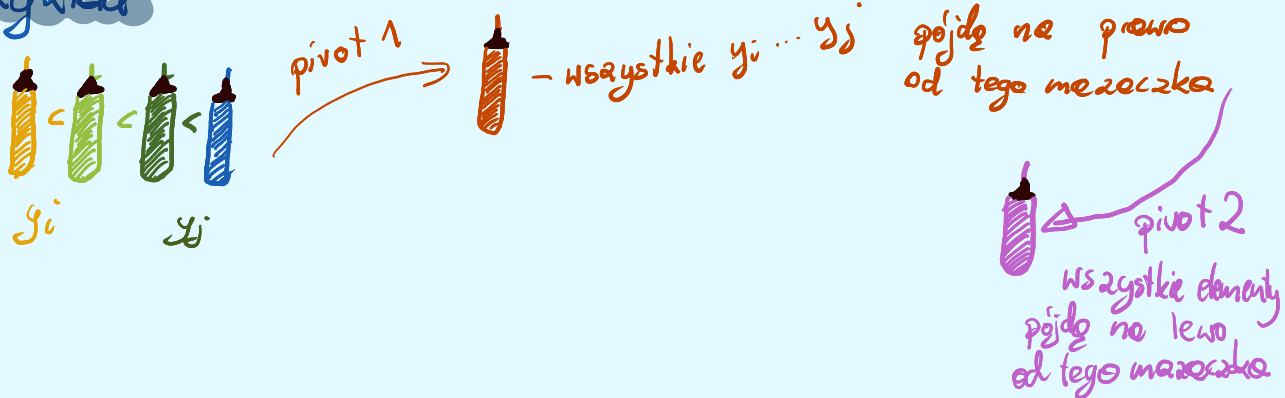
X - liczba porównań

X - macierz porównań
 $x_{ij} = \begin{cases} 1 & - a'_i, a'_j \text{ są porównywane} \\ 0 & \text{wpp} \end{cases}$

$$\mathbb{P}(X_j = 1) = ?$$

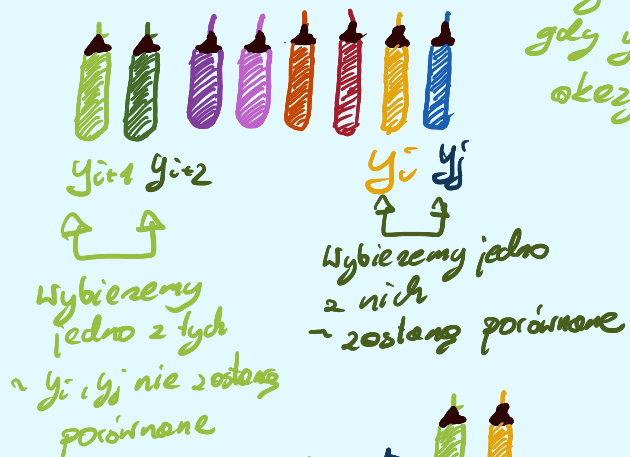
1° Dopóki żaden z elementów a_i^1, a_{i+1}^1, a_j^1 nie stanie się pierwszym $a_i^1, a_{i+1}^1, \dots, a_j^1$ nie będą porównywane ze sobą i będą się znajdować w tej samej podtablicy

Paykital



2° Element y_i i y_j mogą zostać porównane jedynie kiedy jeden z nich jest pivotem. Co więcej mogą zostać jedynie porównane, gdy któregoś z $y_i \dots y_j$ zostało wybrane jako pivot. Jeśli wybierzemy pivotem jako y_i lub y_j to zostaną porównane, jeśli wybierzemy innego, to już nie zostaną porównane, bo trafią do różnych podtablic.

Przykład: Dochodzimy do momentu, gdy jeden z $y_i \dots y_j$ zostaje pivotem. Jest to jedyny moment, gdy $y_i \dots, y_j$ mogą zostać porównane (jedyne okaze, by ktoryś został pivotem).



Zatem szukane prawdopodobieństwo
 jest równe temu, że y_i lub y_j stang
 się pojawi w rozważonym momencie,
 czyli $\frac{1}{j-i+1} \cdot 2^k$ bo zarówno y_i
 jak y_j mogą być pivotami
 tyle jest elementów
 $y_i \dots y_j$

$$= \frac{2}{j-i+1}$$

y_i, y_j nie są
 porównane

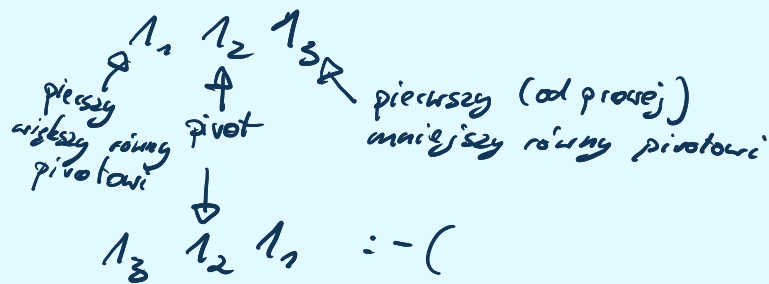
nosz zbiór $y_1, \dots, y_j$
 się podzielił
 a szczególności y_i i y_j
 trafił do innych podzbiórów

2. team X - liczba porównań

$$E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}] = 2 \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{1}{j-i+1} \leq 2 \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{1}{k} \leq 2 \sum_{i=1}^{n-1} O(\log n) \leq O(n \log n)$$

Uwagi:

- Quick sort jest **NIESTABILNY**.



- Mozna zastosować magiczne piótki (odsyłam do notatki o selekcji), ale ze względu na dużą stałą przy $O(n \log n)$ w realnych zastosowaniach jest to niepraktyczne i randomizacja daje lepsze rezultaty.

- standardowo gdy b-e jest mało stosujemy insert-sort - by zbić liczbę wywołań rekurencyjnych.